

Desarrollo de Software 2do Examen Parcial

Desarrollo de Software.

Nombre y Apellido: _____ Nro. Cédula: _____

Fecha: _____

Indicaciones: El examen es individual. Duración sugerida de 120 minutos.
Justifique sus respuestas con criterios técnicos. En selección múltiple puede haber más de una respuesta correcta.

1 SECCION A - TEORIA (2 puntos)

[1.1] En un sistema de ventas, el equipo discute cómo modelar la dirección de envío (ShippingAddress), que incluye calle, ciudad, país y código postal. La dirección se copia en cada pedido y, si el cliente cambia su dirección en el perfil, los pedidos ya registrados no deben cambiar. ¿Cuál es la opción que mejor representa cómo debería modelarse ShippingAddress según DDD?: (1 pto)

- ☐ Como una Entidad propia con un Id, porque aparece en varias tablas.
- ☐ Como un Servicio de Dominio, porque encapsula una regla de negocio.
- ☐ Como un Value Object, porque se identifica por sus valores y puede copiarse sin necesidad de identidad propia.
- ☐ Como un módulo o Bounded Context independiente, porque se relaciona con varios procesos.

[1.2] En una aplicación siguiendo arquitectura hexagonal, se necesita enviar correos electrónicos. El equipo define una interfaz IEmailSender y una clase SmtplibEmailSender que usa una librería SMTP. ¿Dónde deberían ubicarse, típicamente, estos elementos? (1 pto)

- ☐ IEmailSender en el núcleo (dominio o aplicación) como puerto de salida, y SmtplibEmailSender en la capa de infraestructura como adaptador.
- ☐ Ambas clases en la capa de infraestructura, ya que envían correos y eso es un detalle técnico.
- ☐ Ambas clases en la capa de presentación, porque los correos los dispara la interfaz de usuario.
- ☐ SmtplibEmailSender en el núcleo para no depender de la infraestructura.

Desarrollo de Software

2do Examen Parcial

2 SECCION B (8 puntos)

[2.1] Dado el siguiente c/odigo: (4 ptos)

```
export class BillingController {
  pay(amount: number): number {
    if (amount <= 0) {
      return 0;
    }

    const gateway = new StripeGateway("secret-key");
    const logger = new FileLogger("log.txt");

    const result = gateway.process(amount);
    logger.log("done");

    return result ? amount : 0;
  }
}
```

⑦ Identifique y explique cinco violaciones arquitectónicas presentes en el código anterior. Debe mencionar explícitamente a qué principio o criterio violan (por ejemplo: inversión de dependencias, separación de capas, acoplamiento a infraestructura, etc.) y justificar brevemente cada una. (1 pto)

⑦ Proponga una refactorización correcta del diseño anterior. En su respuesta debe indicar (2 ptos):

- ✓ Qué interfaces serían necesarias.
- ✓ Qué elementos pertenecerían a la capa de dominio, a la capa de aplicación y a los adaptadores.
- ✓ Cómo se invertirían las dependencias para que el dominio no dependa de infraestructura.

⑦ Escriba cómo quedaría el controlador después de aplicar la refactorización propuesta. Debe mostrar un método equivalente a Pay que: (1 pto)

- ✓ Reciba las dependencias por inyección de dependencias.
- ✓ Llame a un caso de uso o servicio de aplicación.
- ✓ No acceda directamente a infraestructura ni contenga lógica de negocio compleja.

Desarrollo de Software

2do Examen Parcial

[2.2] Dado el siguiente código (4 ptos)

```
// --- Value Object ---
export class Amount {
  private constructor(public readonly value: number) {
    if (value < 0) {
      throw new Error("Amount cannot be negative.");
    }
  }
  static create(value: number): Amount { return new Amount(value); }
}

// --- Aggregate Root ---
export class Account {
  constructor(
    public readonly id: string,
    private balance: number,
    public readonly isActive: boolean = true
  ) {}

  public getBalance(): number { return this.balance; }

  // Lógica de negocio: Protege la invariante de saldo no negativo
  public withdraw(amount: Amount): void {
    if (!this.isActive) {
      throw new Error("Account is inactive.");
    }
    if (this.balance < amount.value) {
      throw new Error("Insufficient funds.");
    }
    this.balance -= amount.value;
  }
}

export interface IAccountRepository {
  findById(accountId: string): Promise<Account | null>;
  save(account: Account): Promise<void>;
}
```

Desarrollo de Software

2do Examen Parcial

```
export interface IExternalNotificationService {
  // Simula una llamada externa que podría fallar o tardar
  sendNotification(accountId: string, message: string): Promise<boolean>;
}

// --- Servicio de Dominio ---
export class TransactionDomainService {
  constructor(private repo: IAccountRepository) {}

  public async executeWithdrawal(
    accountId: string,
    amount: number
  ): Promise<void> {
    const account = await this.repo.findById(accountId);
    if (!account) throw new Error("Account not found.");

    account.withdraw(Amount.create(amount));

    await this.repo.save(account);
  }
}

// --- Servicio de Aplicación ---
export class AuthorizationAppService {
  constructor(
    private domainService: TransactionDomainService,
    private notificationService: IExternalNotificationService
  ) {}

  public async authorizeAndNotify(
    accountId: string,
    amount: number
  ): Promise<boolean> {
    // 1. Ejecutar la lógica de dominio (retirada)
    await this.domainService.executeWithdrawal(accountId, amount);

    // 2. Llamar a la dependencia externa y verificar si la notificación
    fue exitosa
    const message = `Withdrawal of ${amount} successful. New balance:
[hidden]`;
    const notificationSuccess = await
this.notificationService.sendNotification(accountId, message);

    return notificationSuccess;
  }
}
```

- ⑦ Escriba los tests utilizando las técnicas y buenas prácticas indicadas por su profesor en clases para construir tests robustos, desacoplados y libres de fragilidad. Se le pide hacer los tests para el servicios de dominio y el servicio de aplicación, haciendo uso de Mocks donde se requiera.

3 SECCION C (10 puntos)

Contexto del Dominio: PharmaLogistics S.A. es una compañía de logística especializada en el transporte de productos farmacéuticos de alto valor, muchos de los cuales requieren un control estricto de temperatura (cadena de frío) para mantener su eficacia. El fracaso en mantener la temperatura, aunque sea por un breve período, compromete todo el lote y resulta en pérdidas millonarias y riesgos para la salud pública. El núcleo de su negocio gira en torno a dos conceptos principales: los Envíos (órdenes de transporte) y los Contenedores (unidades de refrigeración activas y monitoreadas)

A continuación se detallan los procesos del negocio:

1. Gestión de Envíos: El sistema debe gestionar Envíos. Cada Envío agrupa uno o más Medicamentos. Cada Medicamento tiene un nombre, un volumen y, críticamente, un RangoTemperatura requerido para su almacenamiento (ej. "Refrigerado", 2°C a 8°C). Un Envío completo debe derivar su propio RangoTemperatura requerido, que se define como el rango más estricto que satisface a todos los medicamentos que contiene. (Ej. Medicamento A [2-8°C] y Medicamento B [4-10°C] resultan en un rango de envío requerido de [4-8°C]).
2. Gestión de Contenedores: El sistema gestiona una flota de Contenedores especializados. Cada Contenedor tiene un VolumenMaximo (capacidad) y un RangoTemperaturaOperativo (lo que el contenedor puede hacer, ej. "Modo Cryo", -25°C a -15°C). Cada Contenedor tiene un Estado (ej. DISPONIBLE, EN_TRANSITO, EN_MANTENIMIENTO, COMPROMETIDO).
3. Sensores: Los contenedores están equipados con sensores que reportan LecturaSensor (compuestas por un Timestamp y una Temperatura exacta) de forma periódica.
4. Proceso 1: Cuando un Envío es creado y confirmado, el sistema debe asignarle un Contenedor adecuado. Un Contenedor es "adecuado" si está DISPONIBLE, su VolumenMaximo es suficiente y su RangoTemperaturaOperativo cubre completamente el RangoTemperatura requerido por el Envío.
5. Proceso 2: Cuando una nueva LecturaSensor llega para un Contenedor específico, esta debe ser registrada. Si esa lectura está fuera del RangoTemperaturaOperativo del propio contenedor, el Contenedor debe cambiar su Estado a COMPROMETIDO inmediatamente. Esta es una regla de negocio inviolable del contenedor.

Desarrollo de Software

2do Examen Parcial

6. Proceso 3: Inmediatamente después de que un Contenedor se marca como COMPROMETIDO (como resultado del Proceso 2), el sistema debe reaccionar. Si ese Contenedor tenía un Envío activo (en estado EN_TRANSITO), ese Envío también debe ser marcado como COMPROMETIDO, ya que la carga se considera perdida.

7. Proceso 4: Antes de que un Envío pueda pasar de PENDIENTE a EN_TRANSITO, se debe realizar una validación final. Esta validación debe (a) confirmar que el Contenedor asignado sigue DISPONIBLE (no fue puesto en mantenimiento a último minuto) y (b) consultar un repositorio externo (IRegulacionesRepository) para asegurar que la ruta planificada es legal para los Medicamentos específicos que se transportan.

Usted debe diseñar el **modelo táctico de DDD**. Modele (puede usar pseudo-código, diagramas UML de clases, o una descripción textual **estructurada y precisa**) los siguientes patrones tácticos, justificando sus decisiones de diseño:

1. Identificar Value Objects y sus invariantes/responsabilidades.
2. Identificar los agregados y sus entidades y sus invariantes/responsabilidades.
3. Identifique los servicios de dominio.